

Lab 5

Title: Design and Implementation of Turing Machine.

Theory:

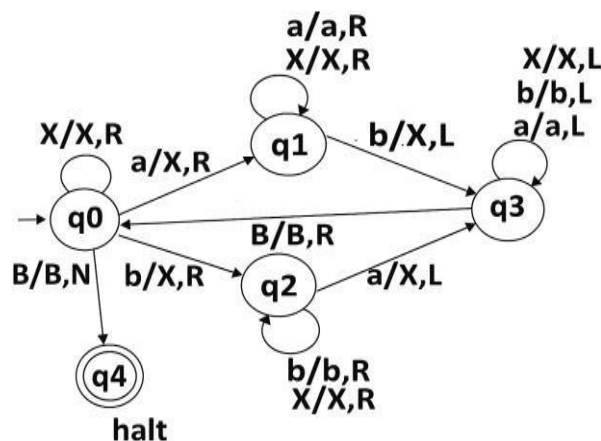
A Turing Machine (TM) is a theoretical model of computation. It's considered the most powerful and fundamental model for a general-purpose computer. Unlike DFAs and PDAs, which have limited memory, a TM has an infinite tape that serves as its memory. This makes it capable of solving any problem that a modern computer can solve, provided there is enough time and memory. The Church-Turing thesis states that a Turing Machine can compute any function computable by an algorithm.

Components of a Turing Machine

A TM is formally defined as a 7-tuple: $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where:

- Q is a finite set of states.
- Σ is a finite set of input symbols (the alphabet).
- Γ is a finite set of tape symbols. This includes all symbols from the input alphabet plus a special blank symbol (B).
- δ is the transition function. This is the core of the TM's logic. It maps a state and a tape symbol to a new state, a new tape symbol to write, and a direction to move the tape head (left or right). Formally, $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$.
- q_0 is the start state.
- B is the blank symbol, used for empty cells on the tape. $B \in \Gamma$.
- F is a set of final (or accepting) states.

Program: Write a C program to simulate a Turing Machine for $L = \{w \in \{a,b\}^* \mid w \text{ has equal number of } a\text{'s and } b\text{'s}\}$



Source code:

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#define MAX 1000

int main()
{
    char tape[MAX];
    int len;
    bool changed;
    printf("Program by: Mahesh Udas\n");
    printf("Enter string over {a,b}: ");

    scanf("%s", tape);
    len = strlen(tape);
    do
    {
        changed = false;
        int i_a = -1, i_b = -1;
        for (int i = 0; i < len; i++)
        {
            if (tape[i] == 'a' && i_a == -1)
                i_a = i;
            if (tape[i] == 'b' && i_b == -1)
                i_b = i;
        }
    }
}
```

```

if (i_a != -1 && i_b != -1)
{
    tape[i_a] = 'x';
    tape[i_b] = 'x';
    changed = true;
}
} while (changed);
bool accept = true;
for (int i = 0; i < len; i++)
{
    if (tape[i] == 'a' || tape[i] == 'b')
    {
        accept = false;
        break;
    }
}
if (accept)
    printf("Accepted: Equal number of a's and b's\n");
else
    printf("Rejected: Unequal number of a's and b's\n");
return 0;
}

```

Output and discussion

```

Program by: Mahesh Udas
Enter string over {a,b}: aaabbb
Accepted: Equal number of a's and b's

```

```

Program by: Mahesh Udas
Enter string over {a,b}: aabbbb
Rejected: Unequal number of a's and b's

```

This C program checks whether a string over {a, b} contains an equal number of a's and b's. It repeatedly finds and replaces one 'a' and one 'b' with 'x' until no more pairs remain. If no unmatched 'a' or 'b' is left, the string is accepted. Otherwise, it is rejected. It simulates a basic Turing machine.

Conclusion:

This lab explored Turing Machine implementation for recognizing strings with equal numbers of 'a's and 'b's over the alphabet {a,b}. The program simulates a TM by repeatedly pairing and marking one 'a' with one 'b' using 'x' symbols until no more pairs exist. The algorithm validates string acceptance by checking if any unmarked characters remain after processing. This implementation demonstrates fundamental TM concepts including tape manipulation, iterative processing, and decision logic. The program effectively highlights how Turing Machines can solve computational problems that require unlimited memory and complex state transitions, illustrating the theoretical foundation of modern computing and algorithmic problem-solving capabilities in formal language recognition.